

AY07: Threading

equipo docente:)



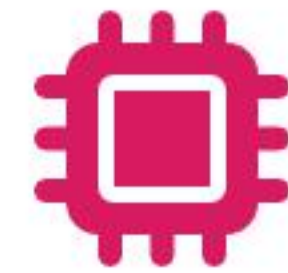
Conceptos Base

Paralelismo, Concurrencia y el rol de los
Threads



Paralelismo

Hardware separado.
Múltiples núcleos físicos.
Ejecución real simultánea.



Concurrencia

Mismo núcleo.
Cambios rápidos de contexto.
Ilusión de simultaneidad.

DEFINICIÓN

Unidad de ejecución dentro de un proceso.



Flujos Simultáneos

Ejecución independiente.



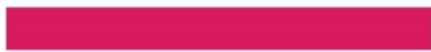
Memoria Compartida

Acceso a mismos datos.



Alta Eficiencia

Ideal para pausas I/O.



Creación y Control

Instanciación, inicio y ciclo de vida

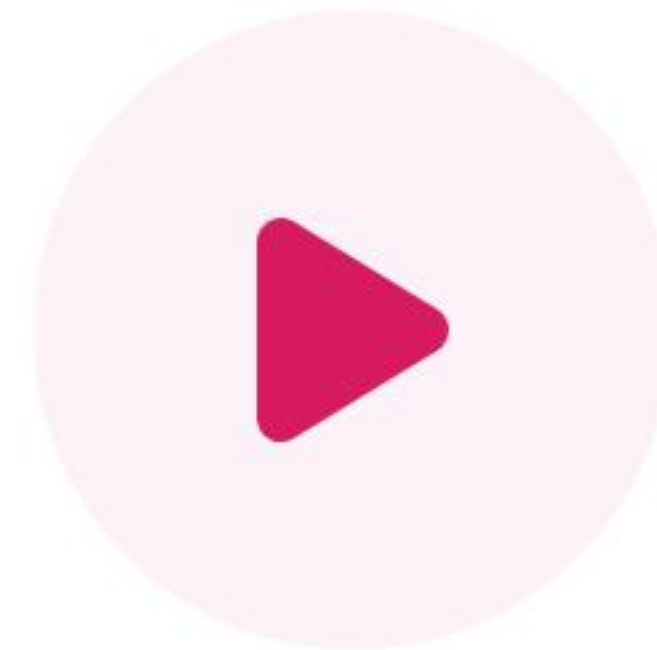
FLUJO BÁSICO

El ciclo de vida de un Thread



Instanciar

Definición del objetivo.



Iniciar

Comienza ejecución.



Esperar

Bloqueo hasta término.

TIPOS ESPECIALES

Daemons y Timers



Daemons

Mueren con programa principal.
Tareas de fondo sin bloqueo.



Timers

Espera programada.
Totalmente cancelables.

SINTAXIS

Creación y Control

`Thread(target=f)`

Crea un nuevo hilo asociado a la función.

`t.start()`

Inicia la ejecución de las instrucciones.

`t.join(timeout)`

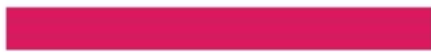
Bloquea el programa esperando al hilo.

`daemon=True`

Configura ejecución en segundo plano.

`Timer(seg, f)`

Ejecuta función tras demora programada.



Sincronización

Recursos compartidos y protección de datos



Condición de Carrera

Múltiples hilos modificando el mismo recurso simultáneamente produce datos corruptos e impredecibles.



LA SOLUCIÓN

Locks (Candados)

- ✓ Llave única para código crítico.
- ✓ Candado bloqueado retrasa otros hilos.
- ✓ Obligatorio compartir la misma instancia.

El Abrazo Mortal

Hilos bloqueados infinitamente esperando la liberación de recursos mutuos.

El riesgo principal de usar Locks (Deadlock).

SINTAXIS

Locks

`Lock()`

Instancia un nuevo candado desbloqueado.

`lock.acquire()`

Solicita la llave bloqueando el acceso.

`lock.release()`

Libera la llave para el resto.

`with lock:`

Adquisición y liberación automática segura.



Señales (Events)

Comunicación y orquestación entre hilos

ORQUESTACIÓN

Comunicación Activa



Esperar

Pausa completa hasta activación.



Avisar

Emisor activa la bandera booleana.



Reiniciar

Apaga la señal para reuso.

SINTAXIS

Events

`Event()`

Crea nueva señal apagada (False).

`e.wait()`

Pausa la ejecución hasta señal activa.

`e.set()`

Enciende la señal a todos los hilos.

`e.clear()`

Apaga la señal.

`e.is_set()`

Retorna estado booleano sin pausar.

¿Preguntas?

Fin de presentación.